

自作暗号化システム

作 中1

目次

1-1 作ろうと思った理由	3
1-2 まず暗号とは？	3
1-3 暗号化システムの仕組み	4
1-3-1 共通鍵暗号方式	4
1-3-2 公開鍵暗号方式	4
2-1 暗号を考える工程	7
2-2 実際に作ってみる	9
3-1 直せていない課題点	11
3-2 実際に作ってみた感想	11
4-1 参考文献	12

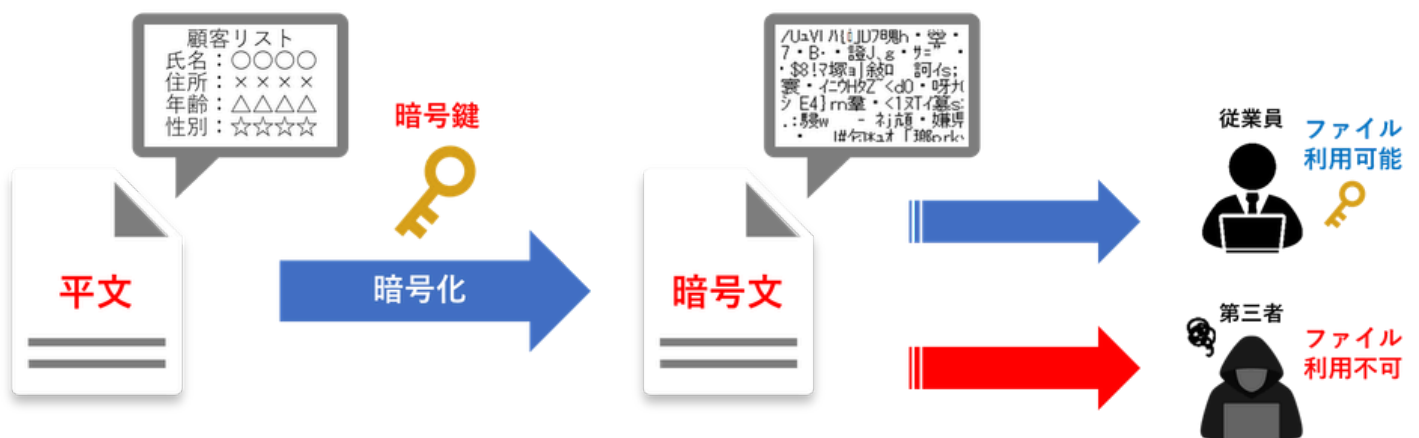
1-1作ろうと思った理由

僕は趣味でwebサイトを作ったりしている。webサイトを作るときにパスワードを安全に保存するためには暗号化しないといけないことを知った。暗号化するソフトは大体お金がかかる。お金がかかるなら本で調べて自分で作ろうと思ったからだ。

1-2まず暗号とは？

データを特定の人しか読めなくするために、一定の手段で変換したデータのことを言う。これを用いた技術を暗号化システムという。データを送信したり保存したりする際に、データを暗号化し、第三者からの盗聴やデータの改ざんを防止するための技術である。

なお、データを暗号にすることを「暗号化」と呼び、暗号化されたデータを元に戻すことを「復号」と呼ぶ。



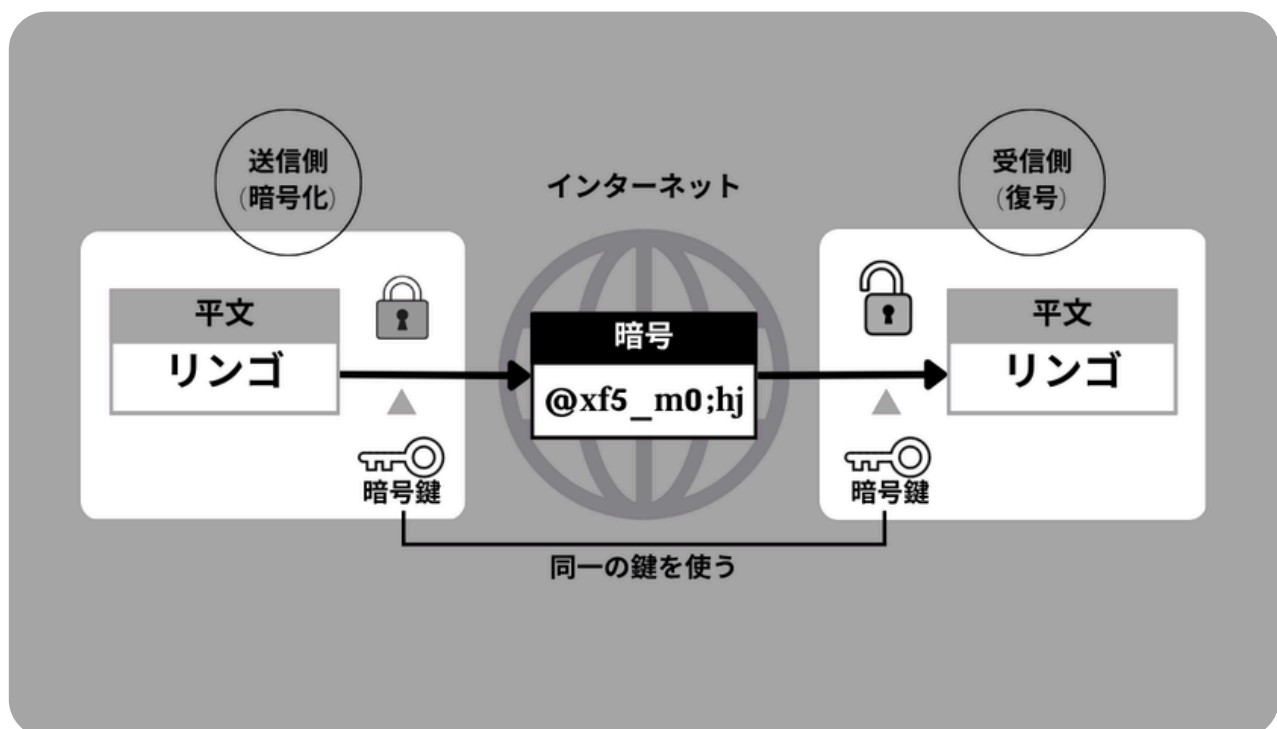
1-3 暗号化システムの仕組み

暗号化システムには共通鍵方式と公開鍵暗号方式というものが存在する。

1-3-1 共通鍵暗号方式

共通鍵暗号方式では、データの暗号鍵と復号に共通鍵という同じ鍵を使用する。共通鍵暗号方式は、同一の暗号鍵でデータを暗号化し、復元する暗号技術だ。

暗号化する側(送信者)が使用した鍵を、復号する側(受信者)も使用する。



共通鍵暗号方式を使用する

有名なアルゴリズム

RC4(アールシフォー)

暗号鍵を元に1ビットずつ疑似乱数を生成し平文とともにXOR演算を行うことで暗号化および復号を行う。

DES(デス、ディーイーエス)

64ビット単位で平文のデータを区切り、その区切りごとの平文を暗号化する。

3DES(トリプルデス、トリプルディーイーエス)

DESからさらに安全性を高めるために考案された暗号技術だ。DESを「暗号」「復号」「暗号」の順で3回実行することで暗号強度を高める。3DESはクレジットカードの暗号化などで、金融関連で幅広く利用されている。

AES(エーイーエス)

平文を先頭から順にブロックと呼ばれる単位に区切って暗号化される「ブロック暗号」と1ブロックごとに暗号化する処理を「ラウンド」とし、鍵長(暗号鍵のデータ量)によって特定回数のラウンドを繰り返すことで暗号化強度を高める。

現在は、共通鍵暗号方式で用いられるアルゴリズムはAESが主流となっている。

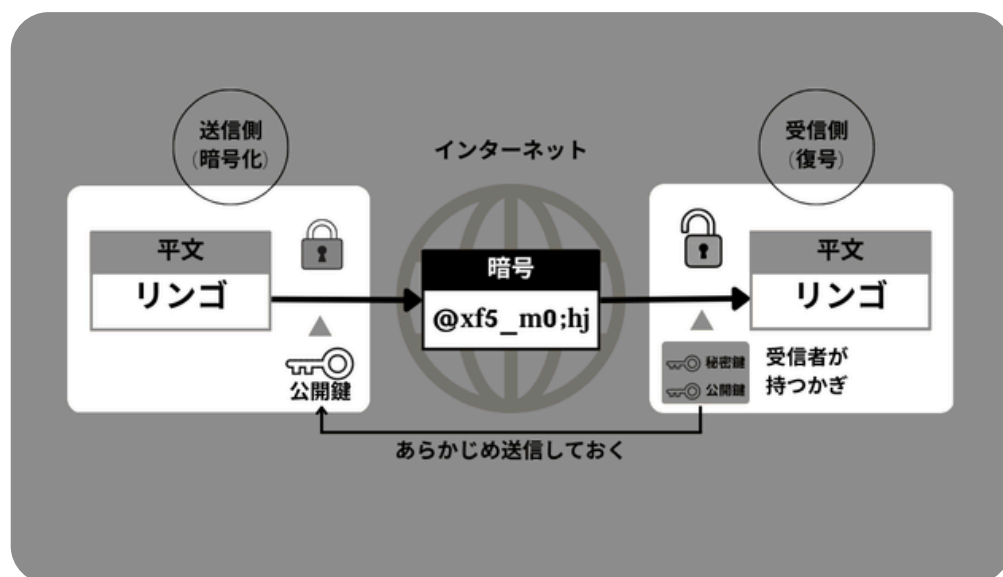
1-3-2 公開鍵暗号方式

公開鍵暗号方式とは、暗号化と復号とで異なる鍵を使うことにより、暗号鍵を盗聴されるリスクを低減した暗号方式だ。暗号化の鍵を公開することからこのように呼ばれている。

公開鍵暗号方式では、送信者と受信者が異なる暗号鍵を保持しているのが特徴で、送信者側は受信者の公開鍵でデータを暗号化して送信し、データを受け取った受信者は受信者の秘密鍵で復号する。公開鍵で暗号化しても復号するには秘密鍵が必要なため、公開鍵が流出しても暗号化したデータを守ることができる。

公開鍵暗号方式の手順

- ①メッセージ受信者が秘密鍵と公開鍵を作成し、公開鍵を展開する。
- ②メッセージ送信者が受信者の公開鍵を取得し、公開鍵を使って通信内容を暗号化する。
- ③暗号化された文書をメッセージ受信者が受け取る。
- ④メッセージ受信者が秘密鍵を用いて復号し、メッセージの内容を確認する。



公開鍵暗号方式の代表例

RSA暗号

RSA暗号の最大の特徴は、中学一年生の数学で学ぶ素因数分解を利用している点だ。これにより高い安全性を担保している。

公開鍵暗号方式は、共通鍵暗号方式の課題であった鍵管理において、鍵の配送および盗聴のリスクに対策する必要がある。

暗号を用いたデータ通信において、共通鍵暗号方式の課題を解決した方式と言える。

ただこれには計算スピードが遅くとても重いという弱点がある。

なのでスマホやPC (Android, iOS, Windows, Mac) の端末内のデータ暗号化、そしてGoogle ドライブのデータ暗号化で使われているのは、すべて「AES」で行われている。

2-1 暗号を考える工程

暗号化するには暗号を考えなければならない。

アルゴリズムが単純な共通鍵暗号システムをまねて自分で暗号方式を考えることにした。

まず一番最初に思いついたのが $a=01$ $b=0a1$ のようにする「単一換字式暗号」という暗号だ。

ただこれだと「文字の登場確率（頻出度）」でバレることに気づいた。

例えば、日本語の文章を暗号化する場合、どんなに文字を置き換えても、変換後のデータには「い」「ん」「か」に該当する文字が大量に登場することになる。

ハッカーなどは、文字の中身が分からなくても、「この暗号文の中で一番多く使われている文字は、きっと『い』だな」と推測（頻度分析）して、パズルのように簡単にルールを特定してしまう。

なので次に入力された文字が ab だった場合 $ab=010a1$ にしてそこから $0101._a$ にして $0101._01$ にしようとした、がこれだと「同じ文字」を入れたときにパターンが見えてしまう。

このルールのままだと、例えば「aaaaa」のように同じ文字がずっと続くデータを入れたとき、最終的な出力も「01,01,01...」と同じパターンの繰り返しになってしまう。

さっきの暗号を複雑にするために $a=.$ $b=..$ などにしようとした。だけどこれだと aaa や ba などでも同じになってしまう。

なのでこのシステムで記号の種類を増やし、どの順番で並んでいるのかをわかるようにした。

まず $a=.$ $b=,_$ $c=,_-$ にして記号をくつつける。

そうすると $.,_,-$ になる。記号の数も数えとく。これだと6個なので6になる。次に記号のアルファベットを決めて、個数の横にアルファベットをくるようにする。 $.=x$ $._=y$ $._-=z$ $._--=w$ これだと $1x2yz2w1$ になる。この前に数えておいた数字を入れる。入れると $6.1x2yz2w1$ になる。

そしてアルファベットを数字にして後ろに持ってくる。

x=1 y=2 z=3 w=4

持ってくると6.1221.1234

復号の仕方

後ろの数字を文字に直す。6.1221.1234=6.1221.xyzw

これを数字にくっつける。6.x1y2z2w1

そして記号にする。この時前にある数字はなくても解読できるので消す。.,,_-になる。

この記号だと-が入っているからcができる。後ろに-があるからcは、一番最後に持っていく。.,_cになる。

次に残りの文字で.が入ってるから.はaしかないのでa,_c残りの記号でできる文字はbだからabcになる。

この復号の仕方だとやり方が曖昧でコードに直しにくいのでプログラミング用語で「再帰」というやり方に変えた。

- 1、最初の一文字目が「,」ならbにして違ったら2へ
- 2、最後の文字を見て「-」だったらcにして違ったら3へ
- 3、左から順に解読する。

つまり「_,-,_」だったら1も2もできなくて3から始める。「_」だからc。

次にcを外して「.,,_」で1も2もできないから3をして「.»だからa。

aを外して「_,_」だから1が適用されるからb。外したのを合わせてcabになるような方法だ。

2-2 実際に作ってみる。

今回はVisual Studio Codeというソフトでpythonというプログラミング言語を使い実際に作ってみた。

初めは、すぐにできるだろうと思っていたが実際は簡単にはできなかった。

やっているうちにあるバグが起こったcabにすると記号は「`_,.,_`」になるこれを数字にすると3241になるこれだと真ん中にあるはずのa(=1)が一番最後にあることになり、解釈をしているとcbaになる。

これを解消するために同じ記号を使わずに`c=¥^-`に変更し、新しく`¥=v ^=u`と`v=4 u=5`を追加した。

こうすることで`¥^-.,_`になり、456123になりaが真ん中にあることになる。

復号前の数字を見ると6.111111.456123になり、真ん中の数字が1だらけになってしまう。これでも良かったが今回は、プログラミング用語で「ソルト（塩）」というランダムな数字を真ん中にかけるテクニックを使った。

例えばランダムな数値が245だとして、111111にかけたら27222195になる。

これを真ん中に入れると6.27222195.456123になる。こうすることで復号を単純化しながら、規則性を見つけにくくすることができる。

バグが直りアルファベットが3個しかなかったのを26個に増やした。

この時に記号を数字に変換するときに二桁の文字があった。これをバグなく処理するためにダミー文字をランダムに挿入した。数字の間にランダムなアルファベットをランダムに数個、入れた。

例えばhelloを暗号化するとh=44になる。このまま数字をくっつけると44165051などになる。

これだと数字を4で解読すれば良いのか44で解読すればいいのかわからなくなるので二桁になる数字の前にはランダム個のアルファベットをランダム個入れるということだ。

例44165051=a44hdj16mvh50jncj51など

この暗号の複合の仕方

入力した文字がhelloの場合数字は

10.906666576.w44spw16bgdz50mx51qc32uob29p32i29ltu29cv30になる。これは前の線を引いてあるところは重要ではなく、残りのアルファベットと数字がまっざっているところが一番重要である。アルファベットを取り除くと44,16,50,32,29,29,30のように数を取り出すことができる。

さらに、プログラム内部では数字と記号を対応表（辞書みたいな物）で管理しているので、この対応表にしたがって変換すると44=~ 16=" \$& 50=\$ 51=& 32=< 29={ 32=< 29={ 30=} となり、これらを合わせると。

~"\$&<{<{}になるこれを文字にしていくと

「-」=h 「\$&」=e 「<{」=l 「<{」=l 「{}」=oになる。

合わせるとhelloと復号することができる。

これで暗号化システムは完成だ。

3-1 直せていない課題点

この暗号は、自分で暗号化と復号ができるように設計し、第三者には簡単に復号をさせないようにはしたが、実際の暗号方式と比べるといくつか課題がある。

例えば、変換ルールが分かると復号できる可能性があることや、ランダムな文字の挿入だけでは十分な安全性を保証できない。

また、文字数が増えると暗号文が長くなり、効率も低下してしまう。

でもこの暗号は友達などと遊ぶぐらいには暗号化システムとしては機能する。

3-2 実際に作ってみた感想

最初は暗号は文字を別の文字に置き換えるだけだと思っていたが、本で調べていくうちに、実際の暗号には頻度分析への対策や鍵の管理など、さまざまな工夫があることを知った。

実際にプログラムを作ってみると、思っていたよりも難しく、何度もバグがおきた。特に、復号すると文字の順番が変わってしまう問題や、同じ記号から始まる文字を正しく判別できない問題があり、そのたびに原因を考えてプログラムを修正した。試行錯誤を繰り返しながら少しずつ改善していくことの大切さを学んだ。

また、自分が作った暗号は独自の方式であり、実際に使われている暗号ほど安全とは言えないが、自分で仕組みを考え、プログラムとして完成させられたことは大きな達成感があった。安全な暗号システムを開発しているプロのエンジニアたちはすごいと改めて実感した。

今後は、秘密鍵や公開鍵暗号などについてさらに学び、自分の暗号システムをより安全で使いやすいものに改良していきたい。

4-1 参考文献

1 「最前線に立つプロが教えるセキュリティの基礎」

著者名 関谷 護

発行所 株式会社日経BP

発行年 2023年12月11日

2 「新・標準プログラマーズライブラリ 試してわかる Python [基礎]入門」

著者名 谷尻 かおり

発行所 株式会社技術評論社

発行年 2021年12月4日

完全自動暗号化システムの 動作環境・対応OS

このシステムは、クラウドサーバー（Streamlit Cloud）で動作するWebアプリケーションのため、端末の機種（PC・スマートフォン・タブレット）やOSを問わず、ブラウザ経由で幅広く対応している。

【対応OS・ブラウザ推奨環境】

iPhone / iPad (iOS / iPadOS) : iOS 15 以上 (Safari / Google Chrome 最新版)

Android スマートフォン : Android 9.0 以上 (Google Chrome 最新版)

Windows パソコン : Windows 10 / 11 (Microsoft Edge / Google Chrome 最新版)

Mac (macOS) : macOS Monterey (12) 以上 (Safari / Google Chrome 最新版)



完全自動暗号化システム